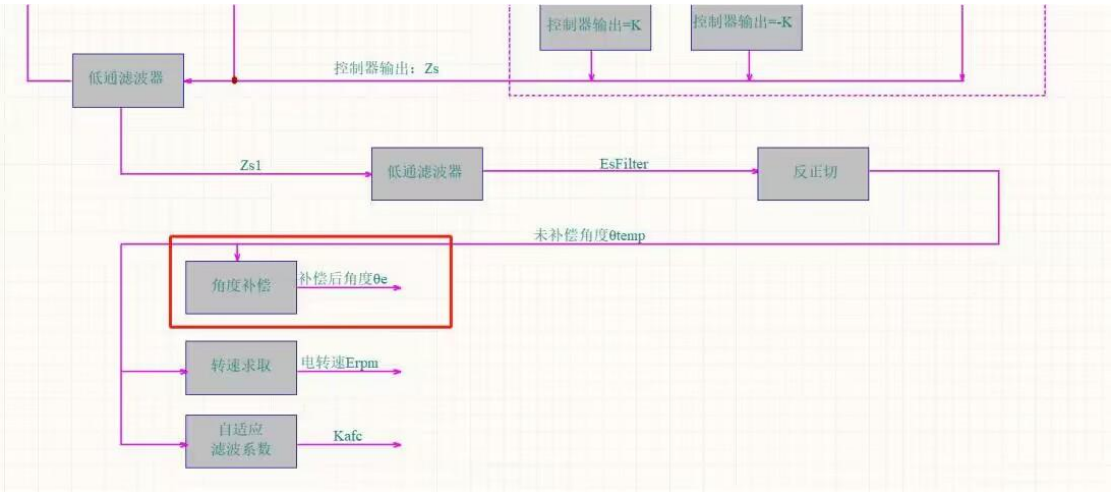
70-FOC 23

大家晚上好，欢迎参加张飞实战电子 FOC 第 11 期学习。
我们接着上次的内容继续啊

```
tObs.qKafc = tObs.qKafcFinal = MulFrac(sTemp,PI_I
tObs.qTheta = qThetaTemp + EANGLE90;
```

我们使用了两次低通滤波，所以这里的相位延迟就是 90°，所以呢我们补偿的话，就补偿 90°。



那么这样的话，补偿的角度也求出来了。接下来，还剩下转速跟滤波系数的求取。我们先来看下估算转速的求取啊。

$$n = \frac{\frac{\Delta\theta}{360°}}{\Delta t}$$

前面呢，我们讲滤波系数求取的时候，讲过这个公式。n 对应的是电转速，Δt 是单位时间，Δθ 是角度变化量。这里呢，为了表示更清晰一些，我们把这个式子中的电转速稍微变化一下。

$$eRPS = \frac{\frac{\Delta\theta}{360°}}{\Delta t}$$

eRPS：电转速，单位转/秒
Δt：单位时间，单位秒
Δθ：角度变化量，单位度

变换之后，我们使用 eRPS 来表示电转速，公式中的这些单位，都如上图所示。但是呢，转速的话，一般是习惯使用转/分来表示的。也就是使用 eRPM 来表示电转速，单位是转/分。因为 1 分钟= 60 秒，所以转换的公式为：

$$eRPM = eRPS*60$$

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

$$eRPS = \frac{\frac{\Delta\theta}{360°}}{\Delta t}$$

$$eRPM = eRPS*60 = \frac{\frac{\Delta\theta}{360°}}{\Delta t}*60$$

所以说，转换后，转速的计算公式变为上图这样了。然后呢，在Δθ/360°这个比值中，Δθ是角度变化量，所以呢，Δθ/360°就相当于是对角度进行归一化。Δθ/360°也就对应0~1之间的小数。在软件中，我们会使用Q15格式来表示小数。例如，在程序中，可以用qDeltaTheta来表示这个归一化值。

$$eRPM = eRPS*60 = \frac{qDeltaTheta}{\Delta t}*60$$

这样的话，求解转速的公式，我们又可以变为上述这样。Δt是单位时间，这个时间呢，由我们自己设定。例如，我们可以取500us，1ms或者其它的值。这个单位时间的不同呢，能够表示的最大转速也就不同，所以，这里符合我们所用的电机要求就可以。若我们取单位时间是1ms，对应就是0.001s，上面这个公式呢，就可以继续变化。

$$eRPM = eRPS*60 = \frac{qDeltaTheta}{0.001s}*60$$

$$eRPM = eRPS*60 = qDeltaTheta*1000*60$$

例如，我们可以来举个例子，假设角度的变化量是 Δθ = 180°。

$$\frac{180°}{360°} = 0.5$$

$$qDeltaTheta = 32768$$

问：0.5的Q15怎么是32768?

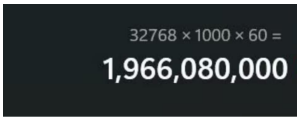
答:这里Q15,和Q16都可以表示这个角度,如果是Q16,那么对应关系就是0~360 对应【0,1），这时180度就表示0.5；如果是Q15就是 -180~+180，对应[-1,+1),当然这时候180度只能表示1了，而不能是0.5了。所以，不管是Q16还是Q15，巧的是180度都是32768。所以这里，有时Q15和Q16都可以解释，而且都是对的。

这样，可以把Q15的角度变化量求出来是 32768，此时呢，我们也可以把对应的转速求解出来。

$$eRPM = eRPS*60 = qDeltaTheta*1000*60$$

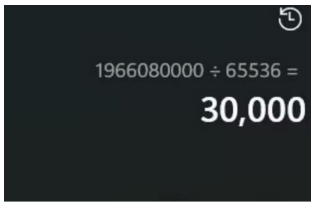
$$eRPM = eRPS*60 = 32768*1000*60$$

但是，实际上我们这里求取的值是扩大了65536倍之后的。所以计算完乘法之后，我们需要把结果除以65536，或者取乘法结果的高16位。



也就是说，我们需要把这个值除以65536，得到实际的转速。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



也就是说，此时对应的电转速是 30000RPM。就是当单位时间内（1ms）的角度变化量是 180° 的时候，对应的转速是 30000RPM。而实际我们电机的最大电转速是 4000RPM（机械转速 2000RPM）。所以呢，我们取单位时间是 1ms，完全可以表示我们的应用场景的。

问：这个是值 1ms 计算一次吗？

答：程序中并不是这样的，因为程序中 1ms 之内的角度变化量是需要经过滤波器的，换句话说，1ms 之内这个角度变化量被滤波了，所以它每个 pwm 周期都在计算。

我们在程序中呢，是把乘法计算拆开来计算的。我们是先计算 32768*1000，然后再把得到的结果乘以 60，而不是像公式中这样连乘。

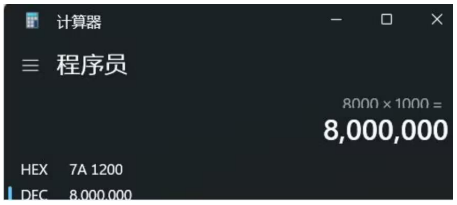


大家看，我们是先计算这个乘法，计算 32768*1000 之后，这个值的十六进制，高 16 位对应 0x01F4，低 16 位对应 0x0000。这个乘法的结果呢，我们使用一个 32 位的变量来存储这里，我们取高 16 位的 0x1F4 来跟 60 相乘，结果正好是 30000。而低 16 位呢，其实可以看成是转速的小数部分，当低 16 位乘以 60 之后的结果大于 65535，对应就会往高 16 位进 1。所以，实际实现的时候，转速我们可以用下面的公式来求解：

$$eRPM = eRPS*60 = qDeltaTheta *1000 *60$$

$$eRPM = Low(\ High(\ qDeltaTheta\ *1000\ )*60\ ) + High(\ Low(\ qDeltaTheta\ *1000\ )*60\ )$$

那我们再来举个例子，帮大家巩固一下。假定 qDeltaTheta = 8000，我们先用 8000 乘以 1000。

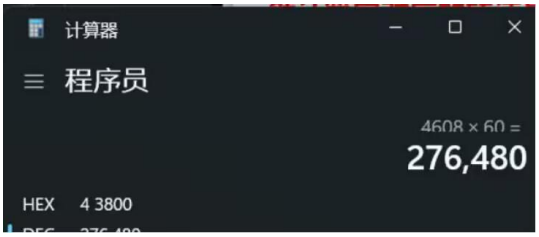


乘完之后，得到的十六进制结果是 0x7A 1200。也就是，这一次乘法计算，结果的高 16 位是 0x7A，低 16 位是 0x1200。然后，分别用这里的高 16 位跟低 16 位乘以 60。0x7A 乘以 60，我们取这个结果的低 16 位，然后 0x1200 乘以 60，取结果的高 16 位，取完之后，把两者的结果相加。



0x7A = 122，所以乘完之后得到 7320。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



然后，0x1200*60，得到的结果是 0x4 3800。我们取这个结果的高 16 位，0x4 跟 7320 相加，就得到 7324。所以，qDeltaTheta = 8000 计算出来，对应的转速就是 7324RPM。

$$eRPM = eRPS*60 = qDeltaTheta *1000 *60$$

本质就是这个公式直接计算，只不过我们程序实现的时候，是分开来计算的。

```
Temp = (unsigned int)tObs.qOmegaFilted;
E_Frequency32.U32 = __builtin_muluu(Temp,1000);
Temp32A.U32 = __builtin_muluu(E_Frequency32.Words.Low,60);
Temp32B.U32 = __builtin_muluu(E_Frequency32.Words.High,60);
tObs.Erpm = Temp32B.Words.Low + Temp32A.Words.High;
```

这个公式，转换为代码，就是这样的。Temp 呢，其实就是对应的单位时间内的角度变化量。第一句话，对应的就是公式中的 qDeltaTheta ，然后，下面的就是按照刚才的步骤来计算，最后呢，取整数部分低 16 位，跟小数部分的高 16 位相加，就得到了对应的值。

这里的乘法呢，使用的就是内建函数。这里呢，变量 tObs.Erpm 中存储的就是估算的电转速了。后面要计算机械转速的话，只需要除以极对数就可以了。这个转速值呢，也是后面我们做速度闭环会使用的。我们计算转速需要用到单位时间内的角度变化量。所以，肯定需要先把角度变化量求解出来，才能求解出转速。

```
qThetaTemp = CalAtan(tObs.qEbetaFinal,tObs.qEalphaFinal);

tObs.qThetaSum += qThetaTemp - tObs.qThetaLast;
tObs.qThetaLast = qThetaTemp;

tObs.qThetaSumCnt ++;
if (tObs.qThetaSumCnt >= IRP_PERCALC)        //1000us / 50us
{
    tObs.qOmega = tObs.qThetaSum;
    tObs.qThetaSumCnt = 0;
    tObs.qThetaSum = 0;
}
```

大家看下这段代码，先把角度求解出来，然后对角度差进行累加。我们取的单位时间是 1ms。函数是 50us 执行一次，所以，就是 50us 累加一次角度差，1ms 的话，也就是需要累加 20 次。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
qThetaTemp = CalAtan(tObs.qEbetaFinal,tObs.qEalphaFinal);

tObs.qThetaSum += qThetaTemp - tObs.qThetaLast;
tObs.qThetaLast = qThetaTemp;

tObs.qThetaSumCnt ++;
if (tObs.qThetaSumCnt >= IRP_PERCALC)        //1000us / 50us
{
    tObs.qOmega = tObs.qThetaSum;
    tObs.qThetaSumCnt = 0;
    tObs.qThetaSum = 0;
}
```

所以呢，这里有一个计数变量。

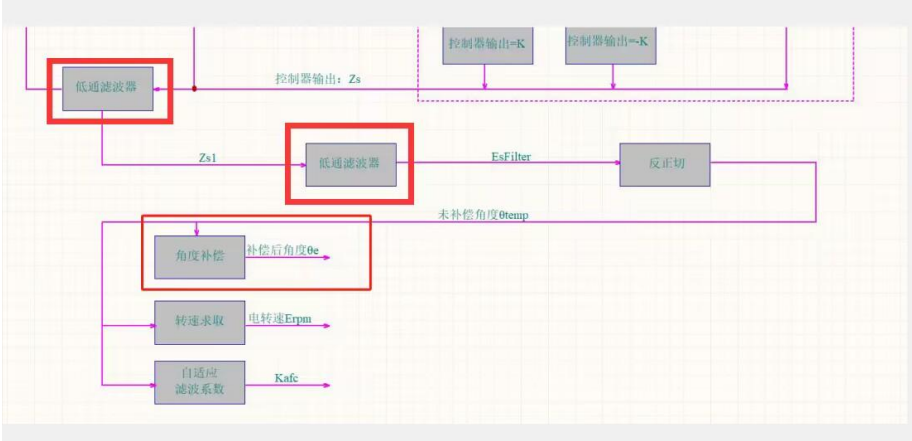
```
/*
#define     IRP_PERCALC                 20      //  1000 / 50
```

也就是累加 20 次，就得到 1ms 内的角度变化量。

```
tObs.qThetaSumCnt ++;
if (tObs.qThetaSumCnt >= IRP_PERCALC)
{
    tObs.qOmega = tObs.qThetaSum;
    tObs.qThetaSumCnt = 0;
    tObs.qThetaSum = 0;
}
```

然后，这个值呢，也就表示单位时间内的角度变化量。理论上呢，我们用这个值去计算转速就行，但是呢，为了让计算结果更稳定些，我们一般会对这个值进行滤波。

红框就是求单位时间，这里是 50us x 20 = 1ms。




```
tObs.qThetaSumCnt ++;
if (tObs.qThetaSumCnt >= IRP_PERCALC)
{
    tObs.qOmega = tObs.qThetaSum;
    tObs.qThetaSumCnt = 0;
    tObs.qThetaSum = 0;
}
```

tObs.qOmega 这个值不属于交流量，所以对这个值的滤波呢，不需要考虑什么相位之后的问题。这里的话，我们可以直接选取一个截止频率，计算出对应的滤波系数，然后调用低通滤波函数去滤波就行。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
tObs.qThetaSumCnt ++;
if (tObs.qThetaSumCnt >= IRP_PERCALC)        //1000us / 50us
{
    tObs.qOmega = tObs.qThetaSum;
    tObs.qThetaSumCnt = 0;
    tObs.qThetaSum = 0;
}
tObs.qKomega = KOMEGAFILTED;
tObs.qOmegaFilted = QLowPassFilter(tObs.qOmegaFilted, tObs.qOmega, tObs.qKomega);
```

这里，滤波系数的定义，是采用宏定义实现的。

```
/*==========================================================
#define      KOMEGAFILTED                    103
```

实际上，这里的滤波系数呢，跟我们之前推导出来的滤波系数公式是一样的。

$$Kafc = \frac{Ts*\Pi}{\Delta t}*qDeltaTheta$$

这个自适应滤波系数是根据 qDeltaTheta 来求解的， 而现在，我们是对 qDeltaTheta 进行滤波。

$$K = \frac{Ts*\Pi}{\Delta t}*?$$

也就是，我们需要确定后面这个截止频率。实际上，这里呢，我们滤波的目的只是想让单位时间内的角度变化量的值更加平滑一些，但是，我们又不想滤波太狠。所以，这里一般这个截止频率，对应取的是支持的最低转速对应的频率值。例如，最低转速是 300，则这里就取 300 转对应的 qDeltaTheta 值参与计算得到的滤波系数。

$$KDeltaTheta = \frac{Ts*\Pi}{\Delta t}* minRpm(qDeltaTheta)$$

也就是，公式变为了这样， 这个时候，我们可以根据之前的公式，来把滤波系数计算出来。当然了，这里说的最低转速，对应的是机械转速，而公式中，使用的是电转速，所以呢，需要把机械转速转换为电转速。

$$RPM*极对数 = eRPM$$

转换的公式呢，也就对应这个。我们电机，极对数是 2，所以如果最低转速是 300 的话，电转速就是 600。

$$eRPM = 600$$

$$eRPM = eRPS*60 = \frac{qDeltaTheta}{0.001s}*60$$

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

$$qDeltaTheta = \frac{600*0.001s}{60} = 0.01$$

$$qDeltaTheta = 0.01 * 65536 = 655$$

然后呢，把确定下来的值，乘以 65535，就可以得到 300 转对应的 qDeltaTheta 值是 655。

$$KDeltaTheta = \frac{Ts*\Pi}{\Delta t} * minRpm(qDeltaTheta)$$

$$KDeltaTheta = \frac{Ts*\Pi}{\Delta t} * 655$$

然后，我们滤波的话，把 655 代入进去，Ts = 0.00005s，π = 3.1415926，Δt = 0.001s。这样的话，滤波系数也就计算出来了，结果呢，是 102.887，也就是 103 了。

```
#define      KOMEGAFILTED                103
```

所以，这个滤波系数就求解出来了。

```
tObs.qKomega = KOMEGAFILTED;
tObs.qOmegaFilted = QLowPassFilter(tObs.qOmegaFilted, tObs.qOmega, tObs.qKomega);

Temp = (unsigned int)tObs.qOmegaFilted;
E_Frequency32.U32 = __builtin_muluu(Temp, 1000);
Temp32A.U32 = __builtin_muluu(E_Frequency32.Words.Low, 60);
Temp32B.U32 = __builtin_muluu(E_Frequency32.Words.High, 60);
tObs.Erpm = Temp32B.Words.Low + Temp32A.Words.High;
```

这个滤波系数求解出来之后，计算好这个滤波后的变量就可以用于转速计算了。

```
qThetaTemp = CalAtan(tObs.qEbetaFinal, tObs.qEalphaFinal);

tObs.qThetaSum += qThetaTemp - tObs.qThetaLast;
tObs.qThetaLast = qThetaTemp;

tObs.qThetaSumCnt ++;
if (tObs.qThetaSumCnt >= IRP_PERCALC)        //1000us / 50us
{
    tObs.qOmega = tObs.qThetaSum;
    tObs.qThetaSumCnt = 0;
    tObs.qThetaSum = 0;
}
tObs.qKomega = KOMEGAFILTED;
tObs.qOmegaFilted = QLowPassFilter(tObs.qOmegaFilted, tObs.qOmega, tObs.qKomega);
```

这段代码把单位时间内的角度变化量滤波值求解出来了。

后面自适应滤波系数的求取，内容比较多，今天可能讲不完，大家觉得需要继续吗？没有人应答，我们就继续了。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

下面讲一下自适应滤波系数的求取。y(n) = y(n-1) + K(x(n) - y(n-1))，这里的 K 是个系数。

$$factor = Ts * 2\pi * fc$$

这是我们使用的一阶数字低通滤波器系数的计算公式。

问：说说， 这种控制方式优缺点，待解决的问题？

答：课程中的算法适应场景是轻负载或者负载稳定的启动场景，比如风机水泵类。超低速和超高速，和变负载会比较困难。

问：二阶数字低通滤波器和一阶数字低通滤波器有什么区别？

答：一阶滤波器的响应相对较为平缓， 二阶滤波器具有更快速的响应速度和更陡峭的频率衰减，适合对信号进行更为精细的控制。但二阶滤波器比一阶滤波器更容易出现振荡。

问：1ms 采集一个数据，然后把采集的数据进行处理，那一阶数字低通滤波器到底滤的什么样的数据？

答：先要分析输入信号的情况，比如如果输入是交流量，本身就有一定的频率，那么滤波的话肯定不希望把自身的原始信号滤除，所以截止频率肯定要大于原始信号频率。如果是直流量，没有交流信号，滤波频率会相对低一些，那么加入了滤波，信号就会有延迟，延迟多少是可以接受的，也是选取截止频率另外的一项依据，希望能对你的理解有帮助。

今天内容就到这里，谢谢大家，大家晚安！

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。